HealthCore

Equipe: Joy

. RM: 573299 - Pedro Vinícius Martins Otenio

. RM : 573616 - Francisco Halison Marques Teixeira

. RM : 572560 – Gustavo santos Viana

1. Objetivo e Escopo do Projeto
2. Descrição das Principais Funcionalidades
3. Tabela de Endpoints da API Restful
4. Modelo de Entidade-Relacionamento (MER)
5. Diagrama de Classes Atualizado
6. Procedimentos para Rodar a Aplicação
7. link do deploy
8. Conclusão

---

1. Objetivo e Escopo do Projeto

O projeto consiste no desenvolvimento de uma API RESTful em Spring Boot 3 para a gestão estruturada de dados de pacientes e médicos no setor da saúde.

O escopo abrange a implementação de uma arquitetura em camadas (`Controller`, `Service`, `Repository`), persistência de dados relacional utilizando o Oracle Database, tratamento de exceções, conteinerização com Docker e o *deploy* em nuvem (Render), viabilizando o consumo remoto por aplicações front-end.

2. Descrição das Principais Funcionalidades

A solução implementa as seguintes funcionalidades principais:

- **Gestão de Pacientes:** Registo completo de pacientes (criação, consulta, atualização de dados cadastrais e remoção), garantindo a integridade das informações clínicas e de contacto.
- **Gestão de Médicos:** Controlo de profissionais de saúde, incluindo o armazenamento de especialidades, identificadores profissionais e dados de contacto.
- **Operações CRUD Completas:** Endpoints padronizados para todas as entidades principais do sistema.
- **Persistência Relacional:** Mapeamento objeto-relacional (ORM) via Hibernate/Spring Data JPA conectado a um banco de dados Oracle.
- **Acesso em Nuvem:** Disponibilização da API em ambiente de produção na nuvem para integração contínua com interfaces gráficas.

Recurso	URI	Verbo HTTP	Descrição	Códigos de Status
Pacientes	`/pacientes`	GET	Lista todos os pacientes registados	`200 OK`, `500 Server Error`
Pacientes	`/pacientes`	POST	Regista um novo paciente	`201 Created`, `400 Bad Request`
Pacientes	`/pacientes/{id}`	GET	Consulta um paciente por ID	`200 OK`, `404 Not Found`
Pacientes	`/pacientes/{id}`	PUT	Atualiza os dados de um paciente	`200 OK`, `400 / 404`
Pacientes	`/pacientes/{id}`	DELETE	Remove um registo de paciente	`204 No Content`, `404 Not Found`
Médicos	/medicos	GET	Lista todos os médicos registados	`200 OK`, `500 Server Error`

Médicos	/medicos	POST	Regista um novo medico	`201 Created`, `400 Bad Request`
Médicos	`/medicos/{id}`	GET	Consulta um médico por ID	`200 OK`, `404 Not Found`
Médicos	`/medicos/{id}`	PUT	Atualiza os dados de um médico	`200 OK`, `400 / 404`
Médicos	`/medicos/{id}`	DELETE	Remove um registo de médico	`204 No Content`, `404 Not Found`

3. Tabela de Endpoints da API Restful

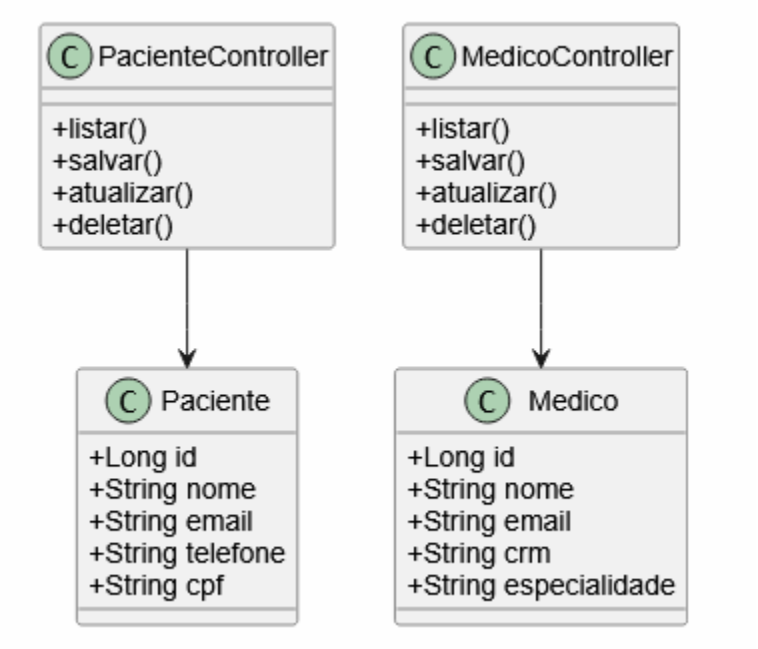
A API expõe os recursos `/pacientes` e `/medicos`, com operações CRUD completas, conforme a tabela abaixo. A URL base é https://sprint4-java-master-att1-1.onrender.com.

4. Modelo de Entidade-Relacionamento (MER)

(Nota: Inserir a imagem do diagrama MER do banco de dados Oracle.)

- **Entidade Paciente:** Contém atributos como `id` (Chave Primária), `nome`, `email`, `telefone` e `cpf`.
- **Entidade Médico:** Contém atributos como `id` (Chave Primária), `nome`, `email`, `crm` e `especialidade`.
- **Relacionamento:** Estruturado de forma a suportar o mapeamento relacional adequado no Oracle Database via Spring Data JPA.

5. Diagrama de Classes Atualizado



O diagrama reflete a arquitetura em camadas do projeto:

- **Pacotes Controller:** Classes de pontos de entrada REST (`PacienteController`, `MedicoController`).
- **Pacotes Service:** Classes de regras de negócio (`PacienteService`, `MedicoService`).
- **Pacotes Repository:** Interfaces de persistência (`PacienteRepository`, `MedicoRepository`).
- **Pacotes Model:** Classes de domínio mapeadas (`Paciente`, `Medico`).

6. Procedimentos para Rodar a Aplicação

6.1 Ferramentas e Requisitos Utilizados

- Ambiente de Desenvolvimento: IntelliJ IDEA / Git Bash
- Linguagem: Java JDK 17
- Gerenciador de Dependências: Maven
- Banco de Dados: Oracle Database

- Conteinerização: Docker

6.2 Passo a Passo para Execução Local

1. Clonar o repositório:

bash
```
git clone https://github.com/Gustavo-S-v/Sprint4-java-master-att1.git
cd Sprint4-java-master-att1
```

2. O projeto está na pasta Sprint4-java-master-att1-main/Sprint4-java, que contém o Dockerfile e o código-fonte (src). O arquivo pom.xml está em src/main/pom.xml.
3. Configurar as credenciais de acesso ao banco de dados Oracle no arquivo application.properties, localizado em Sprint4-java-master-att1-main/Sprint4-java/src/main/resources/application.properties.
4. Executar com Docker, a partir da raiz do repositório (a pasta Sprint4-java-master-att1):

bash
```
docker build -t api-hospital -f Sprint4-java-master-att1-main/Sprint4-java/Dockerfile .
docker run -p 8081:8081 api-hospital
```

A API ficará disponível em http://localhost:8081.

5. Alternativamente, abrir o projeto no IntelliJ IDEA e executar a classe HospitalApplication.

7.LINK DO DEPLOY

https://sprint4-java-master-att1-1.onrender.com/

8. Conclusão

O desenvolvimento deste projeto permitiu consolidar os conhecimentos práticos em engenharia de software e arquitetura backend com Spring Boot 3. A implementação de uma estrutura em camadas organizada, unida à persistência relacional em Oracle Database e ao desdobramento via Docker em nuvem, validou a eficácia da solução. O sistema cumpre `integralmente os requisitos propostos, oferecendo uma API funcional, segura e preparada para ambientes de produção.`